LearnAI:
Self-Adaptive Subject Matter Expert AI Agent System

Sreemaee Akshathala & Amey Karan
2025701028 & 2022111026
Major Project Final Report

November 17, 2025

Contents

1 Introduction **1**

2 Data Collection and Preprocessing **2**
2.1 Data Collection . . . 2
2.2 Parsing Strategy . . . 2
2.2.1 Chunking Schema and Metadata Design . . . 4
2.3 Keyword Extraction and NER System . . . 5
2.4 Knowledge Indexing and Retrieval . . . 5
2.4.1 Embedding Model and Indexing . . . 5
2.4.2 Retrieval Pipeline . . . 6
2.5 Other quality-of-life improvements . . . 6

3 System Overview **7**
3.1 Workflows . . . 7
3.2 Orchestration of a query through the system . . . 9

4 Agents **10**
4.1 Onboarding Agent . . . 10
4.1.1 Initial Adaptive Quiz Implementation . . . 10
4.1.2 Problems Discovered During Testing . . . 10
4.1.3 Design Requirements for a New Adaptive Engine . . . 11
4.1.4 Final Adaptive Algorithm: Ability-Based Online Updating . . . 11
4.1.5 Question Selection: Targeting, Exploration, and Difficulty Quotas . . . 12
4.1.6 Final Classification . . . 12
4.1.7 Validation and Observed Improvements . . . 12
4.2 RAG Agent . . . 14
4.2.1 Classification Components . . . 14
4.2.2 Technical Implementation . . . 14
4.3 Content Generation Agent . . . 15
4.3.1 Components . . . 15
4.3.2 Data flow . . . 15
4.4 Report Agent . . . 15
4.4.1 Architecture and Data Flow . . . 15

4.4.2 Integration and User Experience . . . 15
4.5 Specialised Agent . . . 18
4.5.1 Architecture . . . 18
4.5.2 Integration with system . . . 18

5 User Interface **21**
5.1 User Interface and Experience . . . 21
5.1.1 Progressive Web Application Architecture . . . 21
5.1.2 Dual Connection Modes . . . 21
5.1.3 Chat Interface Design . . . 21
5.1.4 Backend API Architecture . . . 22
5.1.5 WebSocket Protocol and State Management . . . 22

6 Limitations and Failure Cases **23**
6.1 Known System Limitations . . . 23
6.2 Recovery Strategies Implemented . . . 23

A Implementation **24**
A.1 Summary . . . 24
A.1.1 Bonus Features Implemented . . . 25

Chapter 1

Introduction

This report presents the complete implementation of LearnAI, a Subject Matter Expert Multi-Agent system designed for K-12 education domain with specialization in probability. Building upon our mid-term adaptive document processing pipeline, we have developed a comprehensive multi-agent architecture featuring adaptive assessment, intelligent content generation, real-time chat capabilities, and advanced learning analytics. The system demonstrates significant advances in educational technology through the integration of adaptive competency assessment quiz, hybrid retrieval-augmented generation for knowledge access, specialized mathematical problem-solving agents, and a real-time web application interface. Key innovations include a workflow orchestrator that coordinates five specialized agents, profile-based personalization with competence-driven content adaptation, real-time WebSocket communication with mathematical notation support, automated document generation with LaTeX preservation and comprehensive learning analytics with visual reporting capabilities.

Core Problem Statement: Students have different learning paces, competence levels, and knowledge gaps, yet most systems deliver one-size-fits-all content. Our system addresses this through profile-driven adaptation at every interaction level, personalizing the learning for each student's capability

The demo of the working of our Agent SME can be found here

Chapter 2

Data Collection and Preprocessing

Mathematical textbooks present several technical hurdles that standard document processors cannot adequately address. Mathematical notation requires careful preservation of spatial relationships between symbols, particularly for fractions, exponents, and complex equations. Layout complexity varies dramatically between pages, with some featuring dense multi-column text while others contain large diagrams and sparse content. Additionally, different content types within textbooks serve distinct pedagogical purposes - definitions require different processing than worked examples or practice problems. Finally, the token-intensive nature of mathematical content can make large-scale processing prohibitively expensive without careful optimization.

2.1 Data Collection

Our knowledge base is focused on curriculum-aligned content that supports effective educational retrieval. Our source for the demonstration of the full Implementation was NCERT Class 12 Mathematics textbook (complete Probability chapter).

Supplementary data we also collected:

- Practice problems with detailed worked solutions from JEE, SAT, GRE and CAT Books.
- Preparation material PDFs from competitive exam sources like CAT, ETS, GRE, SAT, JEE examinations

Corpus Statistics: Approximately 250 semantically meaningful chunks optimized for educational retrieval. This focused approach ensures high content quality, comprehensive curriculum coverage, and efficient retrieval performance.

2.2 Parsing Strategy

In the Mid Report, we detailed the different options we explored for deciding on the parsing tool. Our approach centers on a self-adaptive processing system that chooses between two distinct pipelines based on document characteristics as illustrated in Figure 2.1. The Marker-based[1] pipeline excels at preserving document structure and mathematical notation, while the OCR-based pipeline provides predictable token

[1] https://github.com/datalab-to/marker

usage and handles dense layouts effectively. This dual approach allows the system to optimize both accuracy and cost depending on the specific document being processed.

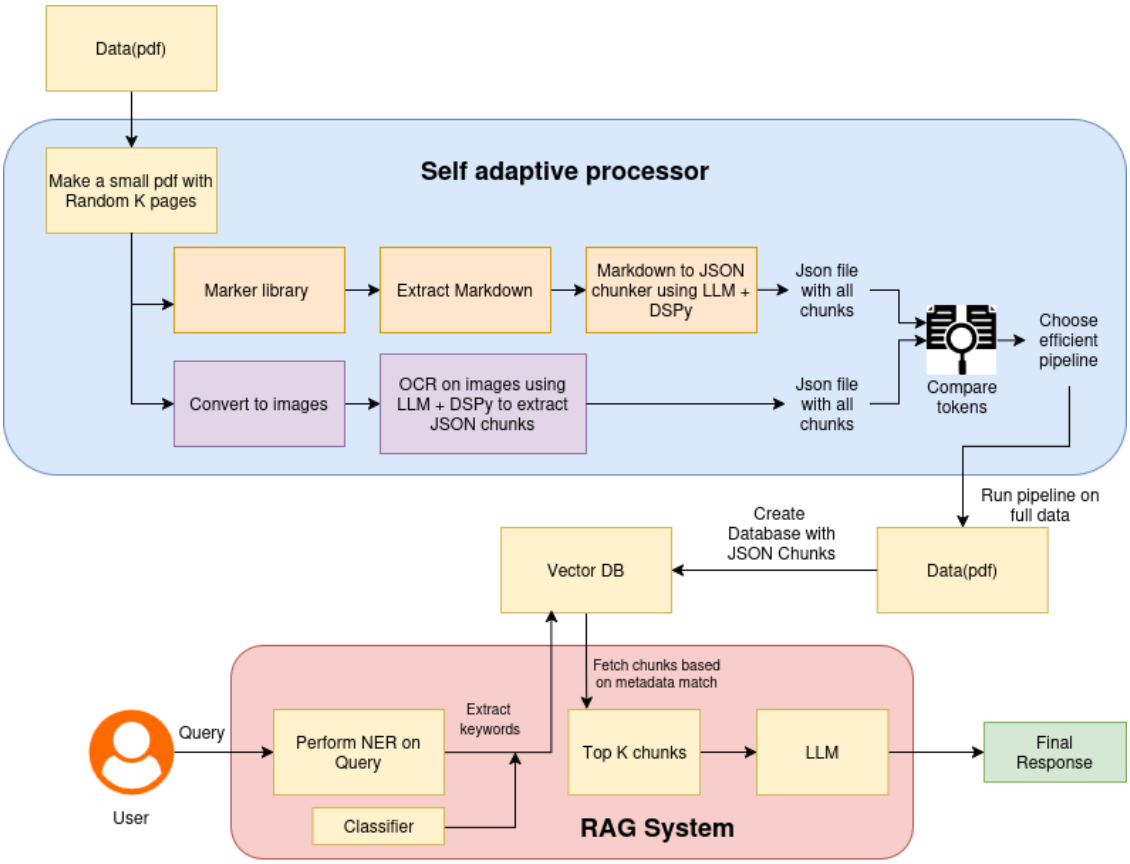


Figure 2.1: Self-Adaptive Parsing System Architecture

The OCR decision was made based on research-grounded recommendations by the 2025 paper *Deepseek OCR*[2]. Their work demonstrated that optical character recognition could achieve remarkable compression ratios while maintaining high accuracy, particularly relevant for our token efficiency goals.

For our NCERT probability textbook, OCR consistently used 888 tokens per page regardless of content density, enabling precise cost calculations. This predictability, combined with the demonstrated reliability for complex layouts, made OCR an attractive complement to traditional parsing methods.

Pipeline Selection Logic

The self-adaptive processor analyzes document characteristics before selecting the optimal processing pipeline. It begins by processing a random subset of pages through both pipelines to establish baseline performance metrics. The system calculates the average token consumption for the Marker pipeline and compares this against the OCR pipeline's consistent 888-token usage per page. Based on this analysis, the processor routes the entire document through whichever pipeline offers better efficiency for that specific content type.

[2] https://arxiv.org/abs/2510.18234

Component Architecture

The Marker-based pipeline processes PDFs directly through the Marker library, which excels at preserving document structure and mathematical notation in LaTeX format. This approach maintains hierarchical relationships and extracts images separately, making it ideal for well-structured documents with moderate content density. However, token consumption varies significantly depending on mathematical complexity and layout density.

The OCR-based pipeline converts PDF pages to images before processing them through Gemini 2.5 Flash with DSPy for structured extraction. This approach provides predictable token usage regardless of content density, making cost estimation straightforward. The pipeline handles dense multi-column layouts effectively but may introduce OCR errors depending on image quality and mathematical notation complexity.

Table 2.1: Performance Comparison: Marker vs OCR Pipeline

Metric	**Marker Pipeline**	**OCR Pipeline**
Avg Tokens/Page	Variable (500-8000)	Fixed (888)
LaTeX Preservation	Excellent	Good
Image Handling	Separate Extraction	Integrated Description
Processing Time	Fast	Moderate
Dense Layout Handling	Moderate	Excellent
Cost Predictability	Low	High

2.2.1 Chunking Schema and Metadata Design

The chunking schema design balances comprehensive metadata capture with practical implementation constraints. Our schema structure accommodates diverse educational content types while maintaining consistency across different document sources as described below:

```
{
    "text": str,
    "metadata": {
        "source_url": str,
        "source_type": str,
        "topics": List[str],
        "subtopics": List[str],
        "chunk_type": Literal[
            "definition", "example", "explanation", "proof",
            "visualization", "practice_problem", "worked_solution",
            "analogy", "misconception"
        ],
        "difficulty_level": Literal["basic", "intermediate", "advanced"],
        "prerequisites": List[str]
    }
}
```

Listing 2.1: Chunking Schema format

Content Type	Description
Definitions	Formal mathematical statements and terminology explanations.
Examples	Concrete applications of abstract concepts.
Explanations	Conceptual understanding presented through narrative description.
Proofs	Logical derivations and mathematical reasoning.
Visualizations	Graphs, charts, and diagrams that support understanding.
Practice Problems	Exercises for student engagement.
Worked Solutions	Step-by-step demonstrations of problem-solving approaches.
Analogies	Connections between new concepts and familiar ideas.
Misconceptions	Common student errors and how to address them.

2.3 Keyword Extraction and NER System

Mathematical terminology requires specialized extraction beyond general NLP capabilities.

Custom NER Model Development:

1. **Training Data Generation:**
   - 500 curated probability-related terms (concepts, theorems, distributions, technical terminology)
   - 2000+ synthetic training examples using templated generation
   - Templates: ”Explain [term]”, ”What is [term]?”, ”How does [term] work?”
2. **SpaCy Model Training:**
   - Training on synthetic examples over multiple epochs
   - Validation on held-out test set
   - **Final Performance: 91% accuracy on mathematical keyword recognition**
3. **Hybrid Extraction Pipeline:**
   - Primary: Custom NER model for precise terminology recognition
   - Fallback: RapidFuzz fuzzy matching for misspellings and partial terms
   - Confidence-weighted scoring prioritizing NER with graceful degradation

This hybrid approach ensures robust keyword extraction across diverse student input styles while maintaining high accuracy on technical mathematical terms.

2.4 Knowledge Indexing and Retrieval

2.4.1 Embedding Model and Indexing

For dense retrieval, we use the **bge-m3** embedding model, selected from the top-performing models on the MTEB leaderboard. Although higher-ranked models exist, **bge-m3** provides strong retrieval performance while producing **1024-dimensional embeddings**, which fits our hardware and memory constraints. This balances accuracy and efficiency.

All documents are embedded during ingestion and stored in a **persistent ChromaDB** instance. The index stores 1024-dimensional embeddings, extracted mathematical keywords, and metadata such as difficulty, content type, and prerequisites.

2.4.2 Retrieval Pipeline

The system uses a multi-stage retrieval procedure combining dense and lexical signals, as required:

1. **Dense Semantic Search:** Query embeddings are compared with stored vectors to identify semantically relevant content. The bge-m3 embeddings improve alignment for mathematical and conceptual queries.
2. **Keyword Filtering:** Terms extracted from the user query act as optional filters or relevance boosts.
3. **Metadata Constraints:** Results are filtered by competence level, content type, and prerequisite fit.
4. **Pedagogical Re-Ranking:** Top-K candidates are reranked using:
    - semantic similarity,
    - keyword relevance,
    - difficulty appropriateness,
    - prerequisite continuity,
    - content diversity.

Using **bge-m3** enables efficient, high-quality vector search while remaining compatible with our hardware. Combined with keyword filtering and educational reranking, this retrieval flow ensures results are both semantically relevant and pedagogically appropriate, fulfilling the RAG retrieval requirements.

2.5 Other quality-of-life improvements

- **DSPy** solved the challenge of reliable structured output generation from language models. Traditional prompt engineering approaches suffered from inconsistent formatting and parsing errors when extracting complex metadata. DSPy's signature-based approach defines clear input-output schemas that automatically generate appropriate prompts for different tasks. The framework enforces type safety throughout the extraction pipeline, reducing errors from malformed outputs. Additionally, DSPy provides robust error handling mechanisms that gracefully manage parsing failures and retry logic.
- Parallel processing architecture addresses the computational demands of large document processing. Documents are segmented into overlapping chunks to preserve context across boundaries, with 500-character overlaps preventing information loss at segment edges. *ThreadPoolExecutor* manages concurrent processing of these segments, significantly reducing total processing time for lengthy textbooks. Result aggregation combines outputs from all segments while maintaining document order. Content-based *deduplication using SHA-256 hashing* eliminates redundant information that appears in multiple overlapping segments.

Chapter 3

System Overview

SME Agent Capabilities:

The system we built has the following capabilities.

Capability	Description
Adaptive Assessment	Algorithm-based onboarding quiz with real-time ability estimation across easy, medium, and difficult levels
Intelligent Tutoring	Step-by-step mathematical explanations with context-aware problem solving using Finetuned LFM2-350M-Math model
Content Generation	Automated creation of PDFs, presentation slides, and worksheets with LaTeX math preservation
Progress Analytics	Learning velocity tracking, topic mastery assessment, and personalized recommendations with visual charts
Interactive Q&A	Real-time chat with mathematical notation support via KaTeX rendering

Table 3.1: Subject Matter Expert Capabilities

We designed the following 5 agents for our SME. The detailed implementation of the agents will be described in Chapter 4.

- **Onboarding Agent:** Algorithm-based adaptive assessment and competence determination
- **Retrieval Agent:** Hybrid RAG system with semantic search and metadata filtering
- **Specialist Agent:** Domain-specific mathematical problem solving with LFM2-350M-Math finetuned model
- **Content Generation Agent:** PDF/PPTX creation with LaTeX math preservation
- **Report Agent:** Learning analytics and progress visualization

3.1 Workflows

We also implemented a workflow orchestration pattern coordinating the five specialized agents through a central orchestrator. This design provides separation of concerns, independent agent optimization, and flexi-

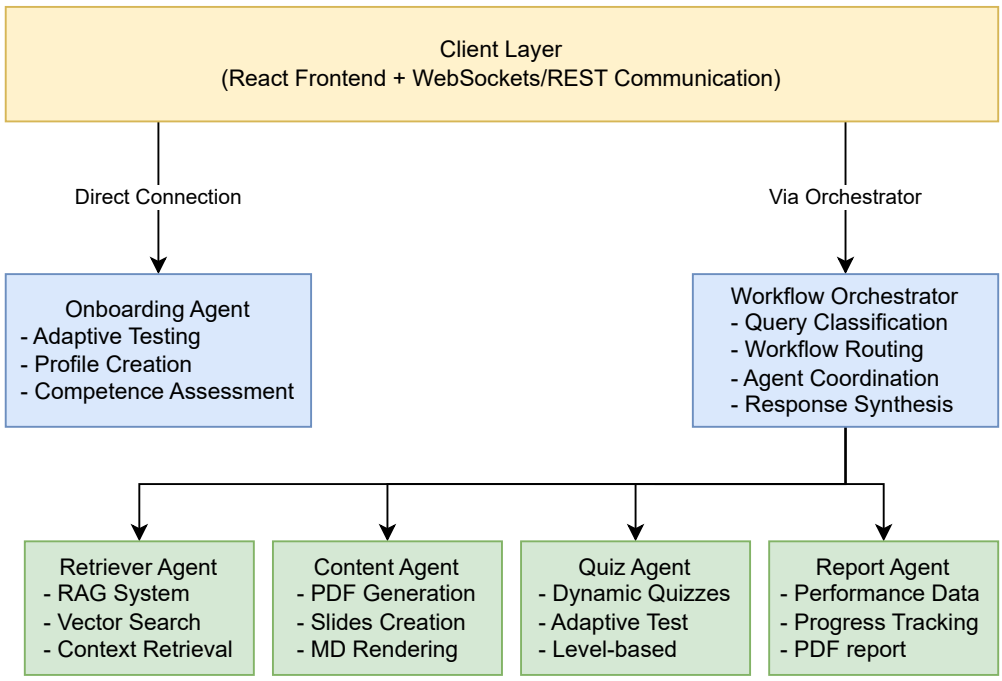


Figure 3.1: Example of complex mathematical notation rendering in chat interface

ble workflow composition. The architecture is a Request-Response model with Sequential Agent Execution. The orchestrator is used for Query classification, workflow selection, agent coordination, and response synthesis.

The orchestrator implements five distinct workflow patterns based on query classification. This allows appropriate agent coordination for different types of educational interactions. These workflows are predefined, but for future work, this can have a dynamic setup such that agents are dynamically selected by the orchestrator for the execution of a task.

Query Type	**Agent Workflow**	**Use Cases**
Concept Explanation	Retrieval	”Explain conditional probability”, ”What is Bayes’ theorem?”
Assessment	Retrieval → Quiz Generation	”Give me a practice quiz”, ”Test my knowledge”
Performance Report	Report Agent (direct)	”Show my progress”, ”Generate analytics”
Domain Query	Retrieval → Specialist	”Solve P(A|B) given P(A∩B)=0.3”, ”Calculate expected value”
Content Creation	Retrieval → Content Generation	”Create a PDF on probability distributions”, ”Make presentation slides”

Table 3.2: Predefined Workflows that the orchestrator can choose from

3.2 Orchestration of a query through the system

For each query, the orchestrator follows this execution model:

1. **Query Analysis:**
   - Extract keywords using custom NER model
   - Classify query type using keyword patterns and intent detection
   - Retrieve user profile for personalization context
2. **Workflow Selection:**
   - Map query type to agent workflow from the available options
   - Prepare combined context object with query and profile data
3. **Agent Execution:**
   - Execute agents in workflow order
   - Pass results from each agent to next as context
   - Maintain intermediate states for multi-step reasoning
4. **Response Synthesis:**
   - Combine agents' outputs into coherent response using LLM
   - Format with appropriate mathematical notation
   - Include generated artifacts (PDFs, charts)

Example Workflow - Domain Query:

- Student Query: "How do I solve P(A|B) if P(A∩B) = 0.3 and P(B) = 0.6?"
- Retrieval Agent: Finds definition of conditional probability, formula P(A|B) = P(A∩B)/P(B), worked examples
- Specialist Agent: Applies formula with given values, generates step-by-step solution with explanations
- Orchestrator: Formats final response with retrieved context and solution

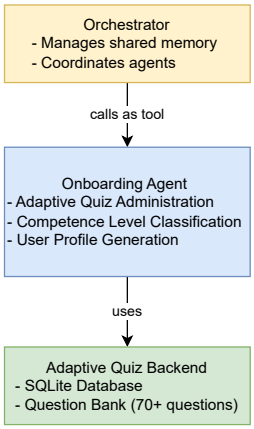


Figure 3.2: Example of complex mathematical notation rendering in chat interface

Chapter 4

Agents

4.1 Onboarding Agent

The onboarding agent represents the first interaction the user would have to be able to use the SME Agent learning system. Its purpose is to accurately estimate a student's competence level before they begin interacting with the orchestration layer. This section documents the complete evolution of the onboarding subsystem — starting from the first working prototype, the problems revealed during real-user testing (human validation by us), and the final redesigned adaptive engine that incorporates ability estimation, exploration, quota enforcement, and robust statistical tuning.

4.1.1 Initial Adaptive Quiz Implementation

Our first implementation followed a traditional rule-based adaptive quiz design. The backend (FastAPI) provided:

- A relational schema containing 100 curated MCQ questions, spanning Grades 7–12, evenly distributed across Easy, Medium, and Difficult levels.
- A question-selection algorithm that always started with Easy questions and promoted the learner to higher levels upon achieving 80% accuracy.
- A fixed 25-question quiz length.
- Automatic competence classification into Easy, Medium, or Difficult.
- A simple correctness-based scoring engine with no continuous ability estimation.

The frontend (NextJS) provided a fully functional quiz interface with real-time progress tracking, feedback, and a performance summary broken down by difficulty level.

4.1.2 Problems Discovered During Testing

Although the first version was fully functional, systematic testing exposed major issues in the adaptivity logic. We conducted multiple simulated runs where we intentionally answered:

1. All answers correctly
2. Almost all answers correctly (e.g., 21/25)

3. All incorrect answers

even with an accuracy of 21/25, the system consistently classified the user as **Easy**. This occurred because:

- Out of the first 18 Easy questions, even a single mistake prevented promotion.
- The system rarely presented Medium or Difficult items, making upward movement nearly impossible.
- The promotion rule (80% threshold per band) was far too conservative.
- There was no exploration mechanism to "test up'' into higher difficulties.
- There was no continuous ability metric; decisions were based entirely on counts.

As a result, almost all users remained in the Easy category, demonstrating that the initial logic failed to differentiate learners with moderate or advanced skill.

4.1.3 Design Requirements for a New Adaptive Engine

Based on these shortcomings, we defined a new set of requirements. The improved system must:

- Preserve the 25-question quiz length.
- Allow strong performers to reach Medium or Difficult early in the quiz.
- Guarantee minimum exposure to higher difficulty levels.
- Estimate ability on a continuous scale instead of discrete thresholds.
- Update ability smoothly after each question.
- Select questions based on both targeting and exploration.
- Make final competence classification based on ability, not raw accuracy.

4.1.4 Final Adaptive Algorithm: Ability-Based Online Updating

The redesigned adaptive engine introduces a continuous ability parameter $\theta \in [0, 1]$ and updates it after every question using a logistic expected-response model. For each question, difficulty levels are mapped to numeric targets:

$$\text{easy} = 0.20, \;\; \text{medium} = 0.50, \;\; \text{difficult} = 0.80.$$

Expected correctness is computed using:

$$P(\text{correct}) = \sigma(\alpha(\theta - d)),$$

with $\alpha = 6.0$ controlling the steepness of the logistic curve.

After the learner responds, ability is updated by:

$$\theta_{\text{new}} = \theta_{\text{old}} + \eta \left(y - P(\text{correct})\right),$$

where $\eta = 0.12$ is the learning rate and $y \in \{0, 1\}$ is the correctness outcome.

This smooth update ensures:

- Correct answers increase ability most when questions are appropriately challenging.
- Incorrect answers reduce ability proportionally and predictably.
- Ability always remains in $[0, 1]$.

4.1.5 Question Selection: Targeting, Exploration, and Difficulty Quotas

The algorithm selects each next question using a combination of mechanisms:

1. **Targeting**: Choose questions whose difficulty score is closest to current ability.

2. **Exploration (Epsilon-Greedy)**: With probability $\epsilon = 0.12$ (decaying to 0.05), choose a question *one level higher* than the current predicted difficulty.

3. **Quota Enforcement**: Guarantee at least:

$$\text{min_medium} = 4, \quad \text{min_difficult} = 2.$$

If the quiz is nearing completion and quotas are unmet, the system forces selection of the missing difficulty levels. This ensures that even high-performing students who make occasional mistakes can still be evaluated at their true level.

4.1.6 Final Classification

At the end of 25 questions, competence level is assigned using:

$$\theta < 0.35 \rightarrow \text{Easy}$$

$$0.35 \leq \theta < 0.65 \rightarrow \text{Medium}$$

$$\theta \geq 0.65 \rightarrow \text{Difficult}$$

This ability-based classification is stable, smooth, and far more representative than the previous accuracy-threshold model.

4.1.7 Validation and Observed Improvements

The new algorithm was tested interactively and through simulation. Sample session traces showed:

- Ability progression reflecting meaningful learning signals (e.g., $0.25 \rightarrow 0.30 \rightarrow 0.39$ after only two correct answers).
- Upward exploration: Medium and Difficult items appearing as early as question 3.
- Faster and more accurate classification for strong learners.
- Elimination of the "stuck in Easy" problem.

The adaptive engine now behaves as a modern, responsive assessment system, providing a robust foundation for all subsequent personalization used by the orchestrator.

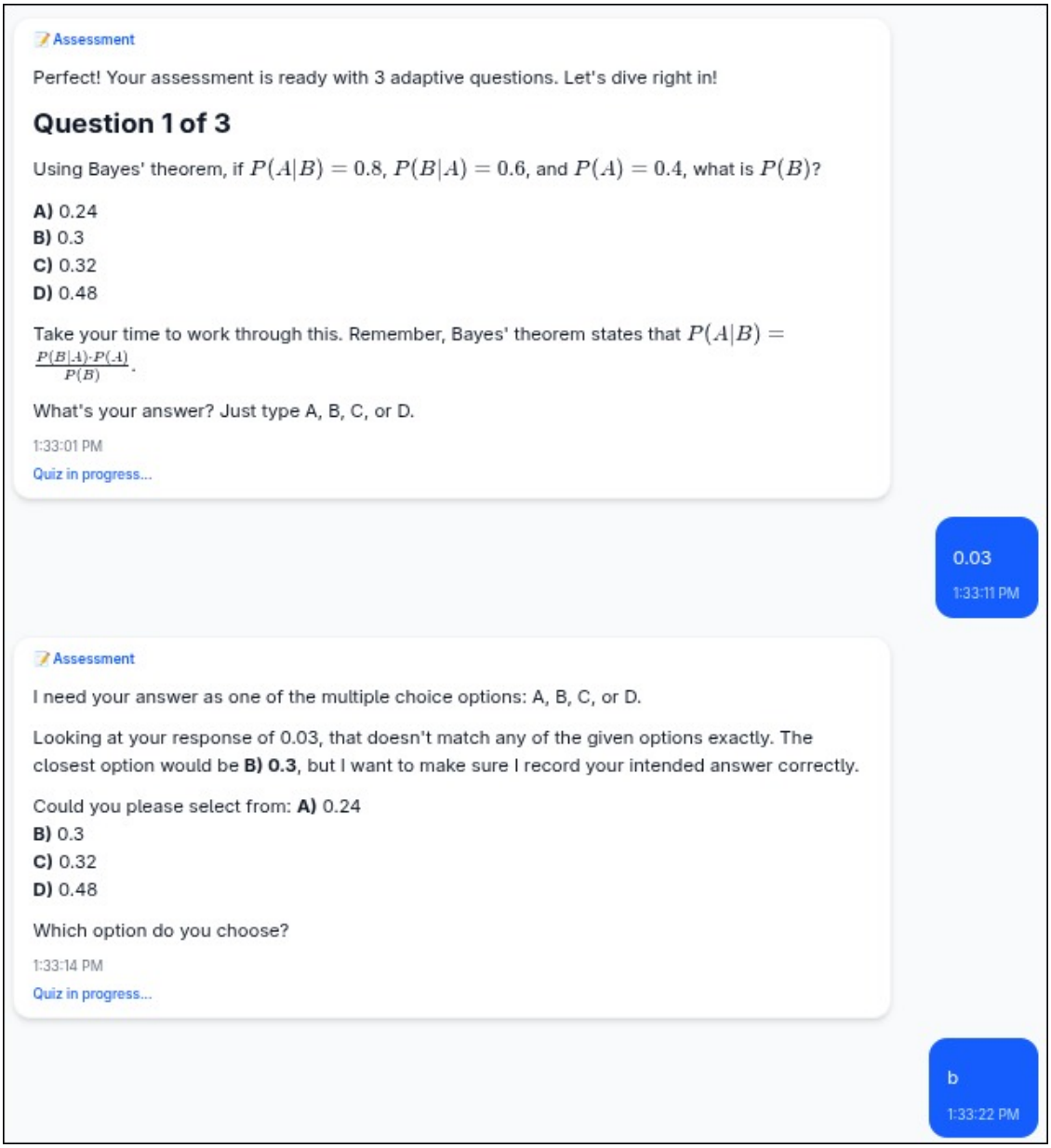


Figure 4.1: Onboarding agent interface showing adaptive quiz progression

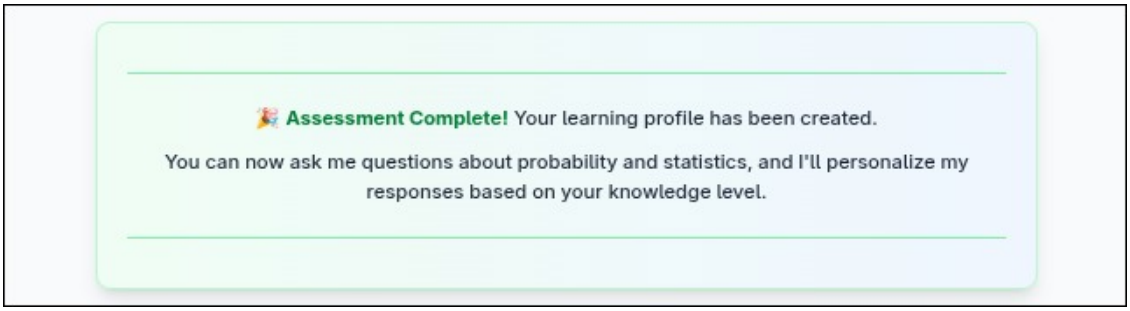


Figure 4.2: Assessment completion screen with final competence determination

4.2 RAG Agent

The RAG implementation integrates multiple retrieval strategies to provide contextually relevant responses. Query processing begins with keyword extraction using our trained NER model to identify mathematical concepts within user questions. Metadata indexing provides structured access to content by educational categories, allowing filtered searches by difficulty level, content type, or prerequisite requirements. Hybrid retrieval combines semantic similarity search with metadata constraints based on the user profile, ensuring both topical relevance and semantic coherence. Relevance scoring considers multiple factors including semantic similarity, metadata alignment, and content freshness to determine final result ordering. Context generation assembles the most relevant chunks while respecting token limits for the final language model input. Response generation leverages this curated context to produce mathematically accurate and pedagogically appropriate answers.

4.2.1 Classification Components

Classification components work together to enrich content metadata automatically. The chunk type classifier analyzes textual patterns and structural cues to categorize content as definitions, examples, or other pedagogical types. Difficulty assessment examines mathematical complexity, vocabulary sophistication, and conceptual depth to assign appropriate difficulty levels. Topic extraction identifies both primary subject areas and supporting subtopics within each chunk. Prerequisite analysis determines what background knowledge students need before engaging with specific content, enabling adaptive learning pathways.

4.2.2 Technical Implementation

Framework: Strands Agent with ChromaDB vector database

- **Vector DB:** ChromaDB with 236+ educational chunks, sentence-transformers embeddings (BAAI/bge-m3)
- **Keyword Extraction:** Custom SpaCy NER model (91% accuracy) trained on 2000+ synthetic examples from 500 probability terms
- **Fallback:** RapidFuzz fuzzy matching for misspellings and partial terms
- **Metadata Schema:** Content type, difficulty level, topics, prerequisites, Bloom taxonomy level, educational objectives
- **Retrieval Strategy:** Hybrid semantic search + keyword matching + profile-based filtering
- **Ranking:** Combined semantic similarity, keyword relevance, difficulty appropriateness, prerequisite continuity

Profile-Aware Filtering:

- Easy competence → Basic & intermediate content only
- Medium competence → Intermediate & advanced content
- Difficult competence → All content with emphasis on advanced

4.3 Content Generation Agent

The Content Generation Agent creates professional documents and presentation slides from markdown content. We built it using the `Strands` framework with three specialized tools that handle PDF documents, presentation slides, and file conversion.

4.3.1 Components

We designed the agent around a core ContentGenerator class that handles all rendering logic. It uses `markdown-it` for parsing, `Jinja2` for templating, and `WeasyPrint` for PDF generation. We wrapped this functionality in three Strands tools: `generate_pdf_document` creates formatted reports with custom styling, `generate_presentation_slides` splits content into slides with gradients and numbering, and `convert_markdown_file` processes existing markdown files. The agent can work in two modes—direct tool calls for programmatic use or through an LLM interface for natural language requests.

4.3.2 Data flow

When a user requests content generation, the agent receives markdown text and output specifications. The ContentGenerator parses any YAML front matter for metadata, converts markdown to HTML, applies CSS styling based on the document type, and renders everything through Jinja2 templates before WeasyPrint generates the final PDF. The whole process takes 1-3 seconds for typical documents. We integrated this with our multi-agent system so students can request ”create a study guide about probability” and the orchestrator coordinates between retrieval for content and generation for formatting.

4.4 Report Agent

The Report Agent tracks student learning over time and generates personalized progress reports. It runs in the background, automatically collecting data from quizzes, study sessions, and learning activities.

4.4.1 Architecture and Data Flow

We built the system with three layers. The data collection layer captures progress snapshots, quiz attempts, topic mastery levels, and learning sessions—everything timestamped and stored in database tables. The analytics layer processes this data through a *ProgressAnalyzer* that identifies trends, calculates consistency scores, and categorizes topics as mastered or needing work. The reporting layer uses six specialized tools to pull analytics and write natural language reports that explain trends, highlight strengths and gaps, and suggest actionable next steps.

4.4.2 Integration and User Experience

We connected the Report Agent to the orchestrator so it automatically handles questions like ”How am I doing this week?” It determines which analytics to pull, retrieves data for the requested time period, and generates responses in a friendly, encouraging tone. We implemented multiple formats—comprehensive

reports, quick summaries, focused analyses, and period comparisons—all available for custom time ranges from days to months.

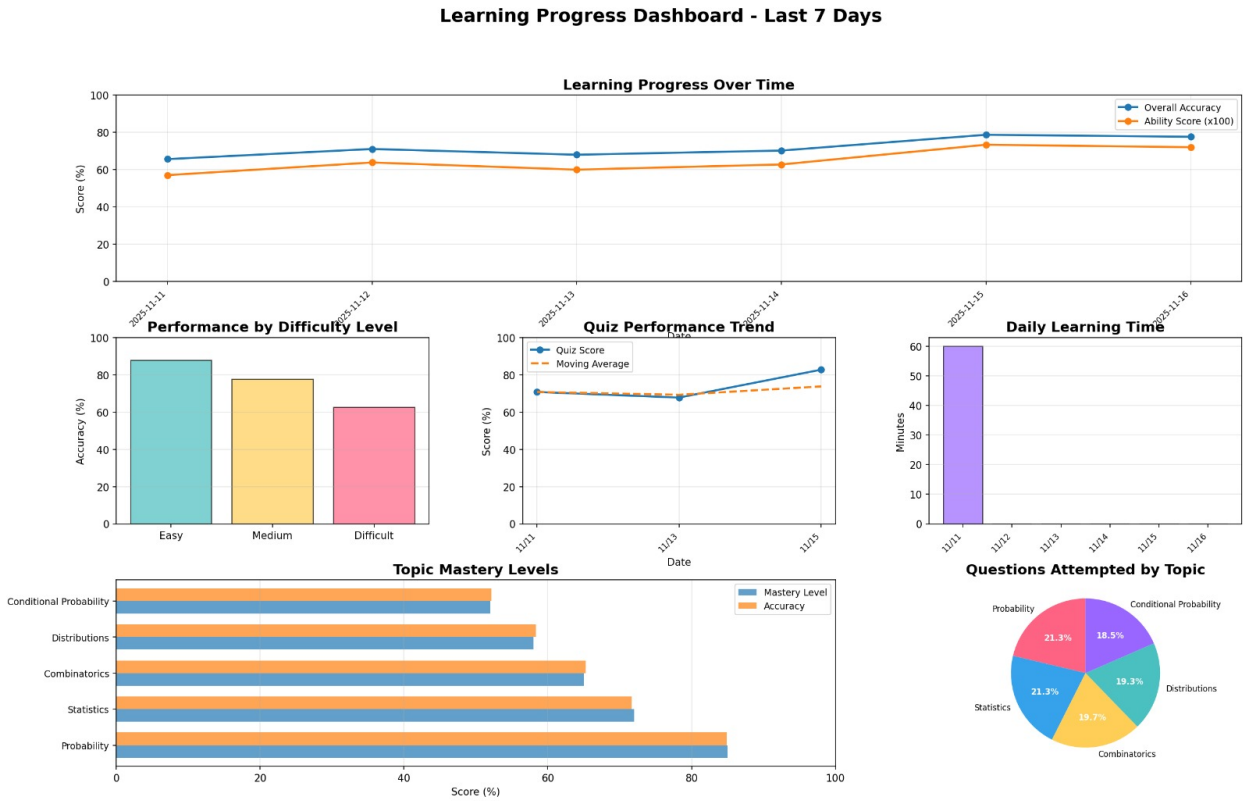


Figure 4.3: Comprehensive learning analytics from the report agent

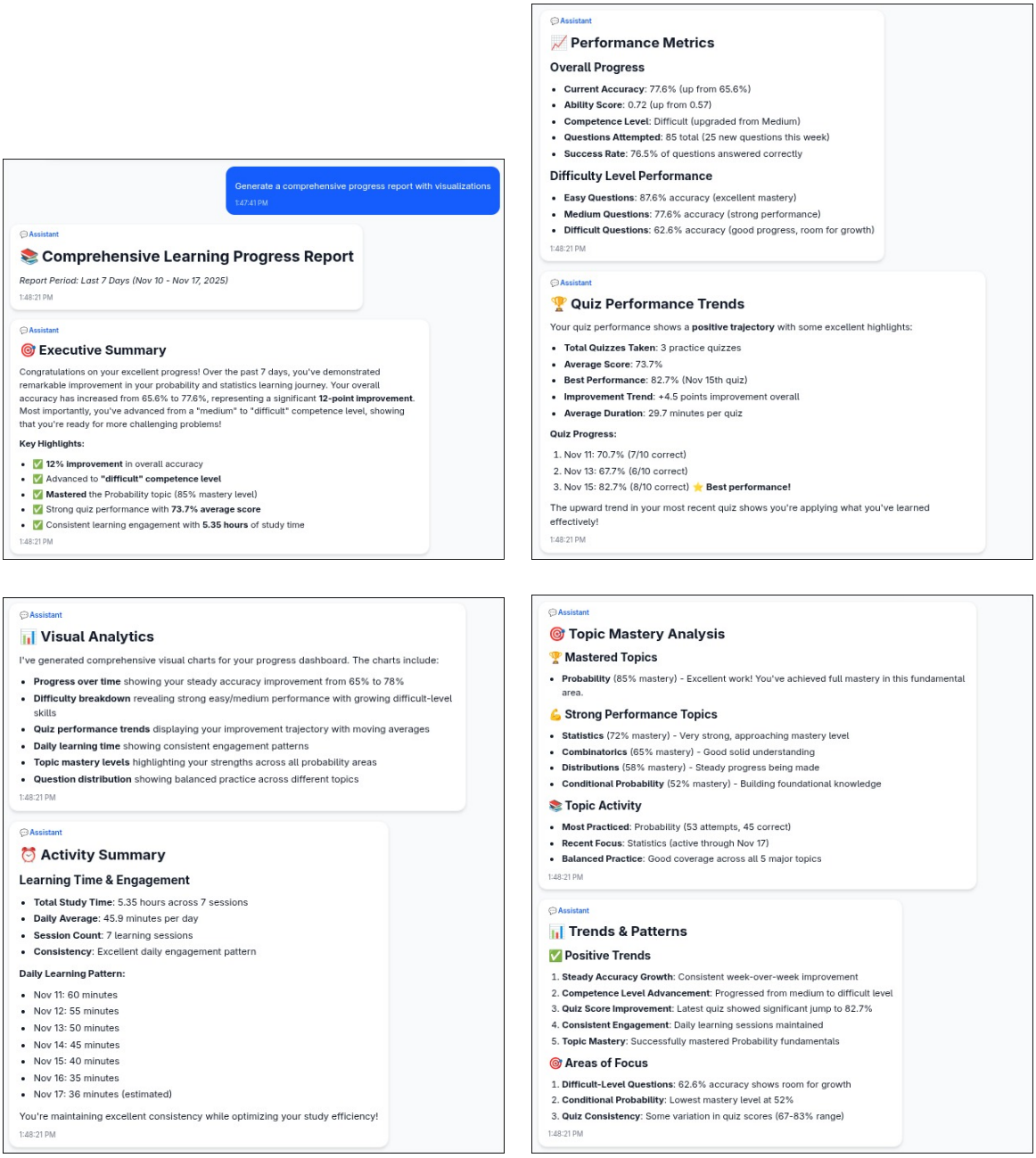


Figure 4.4: Report agent conversation

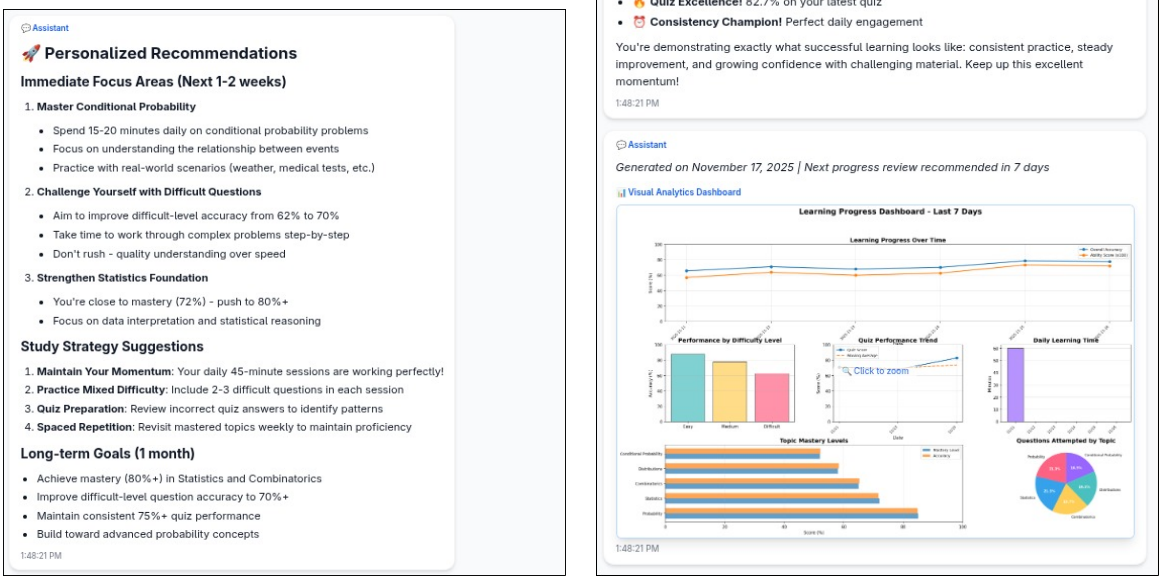


Figure 4.5: Report agent conversation (contd.)

4.5 Specialised Agent

The Specialist Math Agent is a domain-specific problem solver we built using the LiquidAI LFM2-350M-Math model. It handles advanced mathematical problems with step-by-step solutions, integrated into our multi-agent system through the Strands framework.

4.5.1 Architecture

We structured the agent with three main components. The agent core wraps a HuggingFace transformer model and implements the Strands interface, using lazy loading so the model only downloads once (1.4GB) and gets cached for subsequent calls. We built two specialized tools: `solve_math_problem` handles problem-solving with optional context from the RAG system, while `verify_solution` checks proposed answers and provides feedback. The configuration layer centralizes model parameters and defines the agent's behavior as an expert math tutor that explains reasoning step-by-step.

4.5.2 Integration with system

We connected the specialist to our orchestrator so it works alongside the retrieval agent. When a student asks a math question, the classifier identifies it as a domain query, the orchestrator fetches relevant examples or theory from the RAG system, and the specialist uses that context to solve the problem with detailed explanations. The agent excels at algebra, calculus, probability, geometry, and word problems—anything requiring multi-step mathematical reasoning.

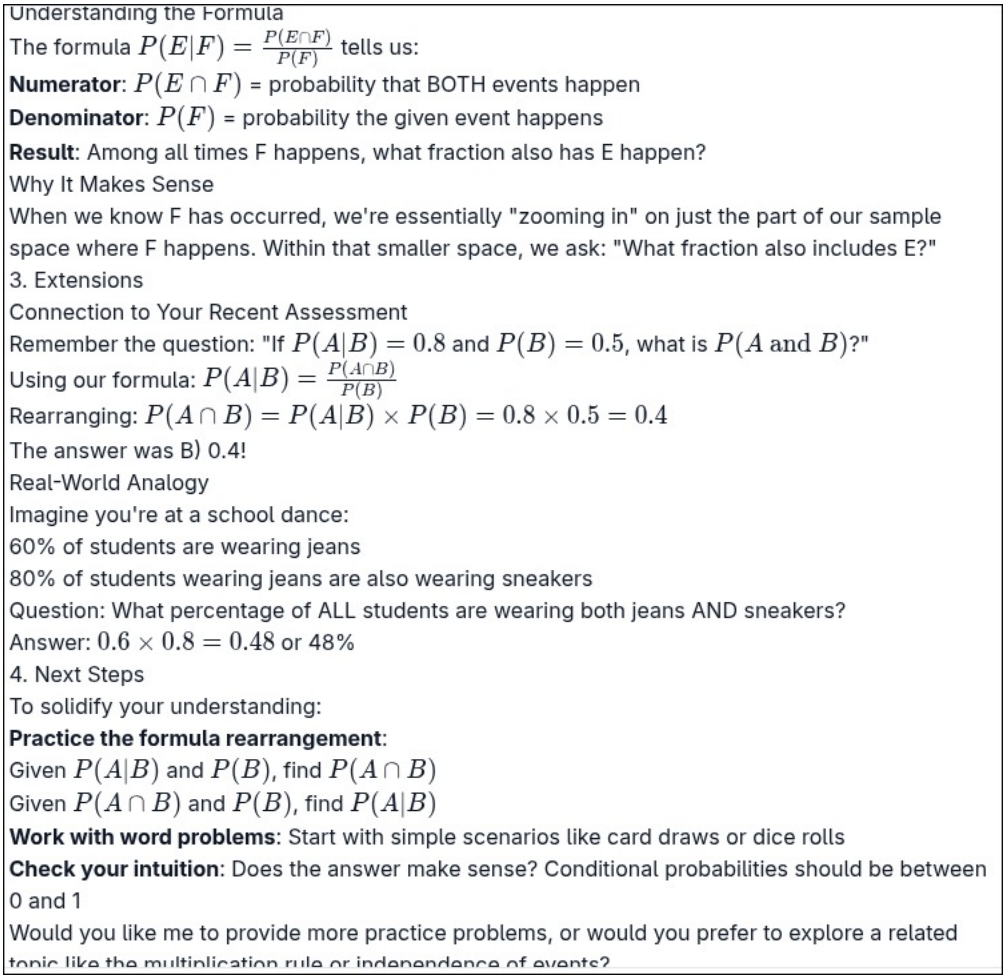


Figure 4.6: Example of complex mathematical notation rendering in chat interface



Figure 4.7: Specialized agent response showing step-by-step mathematical solution

Chapter 5

User Interface

5.1 User Interface and Experience

5.1.1 Progressive Web Application Architecture

We built the frontend using NextJS 14 with TypeScript for type safety, Tailwind CSS for responsive design, and KaTeX for mathematical notation rendering. The system uses WebSocket connections for real-time communication with automatic reconnection handling, while React hooks and context providers manage global state across the application.

5.1.2 Dual Connection Modes

The interface operates in two distinct modes based on user status:

Mode 1: Onboarding (Direct Agent Connection)

New users connect directly to the Onboarding Agent, bypassing the orchestrator to reduce latency during the initial competence assessment. The streamlined quiz interface displays questions with multiple choice options, tracks progress in real-time, and provides immediate feedback on correctness and ability score updates. Once assessment completes, the system automatically transitions to orchestrator mode.

Mode 2: Learning (Orchestrator-Mediated)

After onboarding, users interact through the workflow orchestrator for multi-agent coordination. The chat-based interface maintains message history and context, automatically classifying queries and routing them to appropriate agents while delivering profile-aware responses and generated content.

5.1.3 Chat Interface Design

We implemented a familiar messaging layout with student messages right-aligned and agent responses left-aligned. The rich text editor supports LaTeX notation entry with real-time preview and a quick-insert toolbar for common mathematical symbols. Responses render progressively with streaming chunks, displaying

mathematical expressions through KaTeX, syntax-highlighted code blocks, and properly formatted lists and tables. Generated charts, images, PDFs, and presentations appear inline with download buttons.

KaTeX handles publication-quality mathematical typesetting including fractions, Greek letters, summation and integral operators, probability notation like P(A|B), matrices, and calculus expressions.

5.1.4 Backend API Architecture

We built the FastAPI backend with RESTful endpoints and WebSocket support for all system functionality:

Endpoint	Method	Purpose
/api/sessions	POST	Create session, check profile, initialize mode
/api/chat	POST	Process messages through agents
/api/profile	GET	Retrieve user profile and history
/generated/{resource_id}	GET	Fetch generated content like report or slides
/ws/{session_id}	WebSocket	Real-time communication

Table 5.1: API Endpoints

5.1.5 WebSocket Protocol and State Management

The WebSocket connection flow validates the session, registers the connection, confirms with current state, and begins bidirectional messaging. We handle user messages with query content and context, stream agent responses in chunks with metadata, send status updates for processing indicators, and deliver generated content with document IDs and chart data.

Session management uses an in-memory dictionary mapping session IDs to user data, timestamps, current mode, agent states, and conversation history. We periodically serialize to the database with immediate writes for profile updates, and automatically clean up abandoned sessions after timeout. Agent states persist across queries—retrieval context remains available, specialist maintains intermediate results for multi-turn problem solving, report analysis accumulates during generation, and conversation context uses a sliding window of the last 10 exchanges.

Chapter 6

Limitations and Failure Cases

6.1 Known System Limitations

Through extensive testing, we identified several scenarios where our system exhibits limitations:

Mathematical Complexity Boundaries:

- The specialist agent struggles with multi-step proofs requiring more than 5 intermediate steps, often losing logical thread in longer derivations
- Integration problems involving non-standard substitutions may produce incomplete or circular reasoning

RAG Retrieval Failures:

- Queries using non-standard mathematical terminology or notation variants sometimes fail to retrieve relevant content despite semantic similarity
- Ambiguous queries without sufficient context may retrieve content from wrong topics due to keyword overlap

Content Generation Edge Cases:

- Presentation slides with dense mathematical content sometimes require manual layout adjustments

Onboarding Agent Limitations:

- Students who randomly guess answers may be misclassified due to luck in early questions
- Questions requiring diagram interpretation are text-only, limiting assessment accuracy for visual learners

6.2 Recovery Strategies Implemented

For identified failure modes, we implemented several mitigation strategies:

- Fallback prompting with simplified notation when specialist fails
- Query reformulation when initial RAG retrieval yields low confidence scores
- Graceful degradation to simpler document formats when complex generation fails
- Explicit uncertainty communication when system confidence is below threshold

Appendix A

Implementation

A.1 Summary

Here are the requirements of the project we have satisfied and the bonus sections we attempted:

Table A.1: Component-wise Implementation Details

Component	Implementation Details
Domain Definition & SME Design	Learning Assistant domain clearly defined. SME capabilities include adaptive assessment, mathematical problem solving, content generation, and RAG-based Q&A for queries related to probability.
Data Collection & Organization	High-quality NCERT probability content; supports PDF (Marker) + OCR (Gemini 2.5); structured metadata (topics, difficulty, chunk type).
Preprocessing & Chunking	Adaptive pipeline (Marker/OCR), LaTeX preservation, deduplication, semantic chunking, cost optimization (15–30%).
Embeddings & Indexing	BAAI/bge-m3 + ChromaDB persistent storage, Reranking based on custom chunk metadata
RAG Retrieval	Dense search w/ metadata filters, Hybrid vector + metadata filtering.
SME Capabilities	Mentioned in Table 3.1
Agent Workflow & Tools	Orchestrator with LLM Classifier-based routing, Workflows, UserProfile memory, retry logic, Strands-based coordination, tool integration
Prompting Strategy	Custom prompts per agent with few-shot examples and tuned system messages, DsPY based structured prompts
Document Generation	PDF via WeasyPrint, PPT via HTML/CSS → PDF, markdown → DOCX, KaTeX math support.
System Architecture & APIs	FastAPI REST + WebSocket, modular agents, AWS Strands, Swagger/OpenAPI docs.
Custom NER Model	spaCy NER w/ 500+ probability keywords, 91% accuracy, fuzzy fallback.
Real-time Chat Interface	NextJS + TypeScript, WebSocket live messaging, KaTeX rendering.

Component	Implementation Details
Profile-based Personalization	UserProfile-driven retrieval, personalized recommendations, progress tracking.
Multi-Agent Orchestration	Classifier routing + shared memory across agents; coordinated workflows.
Content Generation Tools	Professional PDFs, PPT slides, LaTeX math rendering.

A.1.1 Bonus Features Implemented

- **Self-Learning & Adaptation:** We implemented difficulty-adaptive question selection using IRT algorithms, built an adaptive quiz system in the onboarding agent that dynamically computes competence levels, and created a SQLite question bank for efficient storage and retrieval.
- **Advanced Analytics:** Developed a comprehensive analytics system that recognizes learning patterns, tracks improvement velocity over time, assesses topic-level mastery, and generates personalized study recommendations based on individual performance data.
- **Real-time Interface:** We built a Progressive Web App with WebSocket communication for instant bidirectional messaging, streaming response delivery for better user experience, and integrated KaTeX for rendering complex mathematical notation.
- **Specialized Models:** Integrated the LiquidAI LFM2-350M-Math model as a domain-specific reasoning engine that handles advanced mathematical problems including algebraic manipulation and proof construction.
- **Content Safety Guardrails:** We added guardrails in system prompts so agents politely decline requests outside their scope (e.g., ”Sorry, I can’t help you with that”) and filter inappropriate content through profanity checks and content sanitization.
- **BGE Reranking:** Beyond basic semantic similarity, we implemented intelligent reranking of retrieved content based on prerequisite continuity, pedagogical diversity, and alignment with identified student weaknesses.
- **Batch Ingestion Pipeline:** We created an automated pipeline for processing and ingesting educational content in bulk, streamlining the addition of new learning materials to the vector database.
- **Session Management:** Built session tracking that maintains conversation history, preserves agent state across interactions, and enables seamless continuation of learning sessions.
- **Report Generation:** Added an analytics engine that generates detailed progress reports, creates performance visualization charts tracking user improvement over time, and provides insights into learning behaviors and outcomes.